hashlock.

Security Audit

Loadout 2nd (dApp)

April 2026 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	13
Centralisation	22
Conclusion	23
Our Methodology	24
Disclaimers	26
About Hashlock	27

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

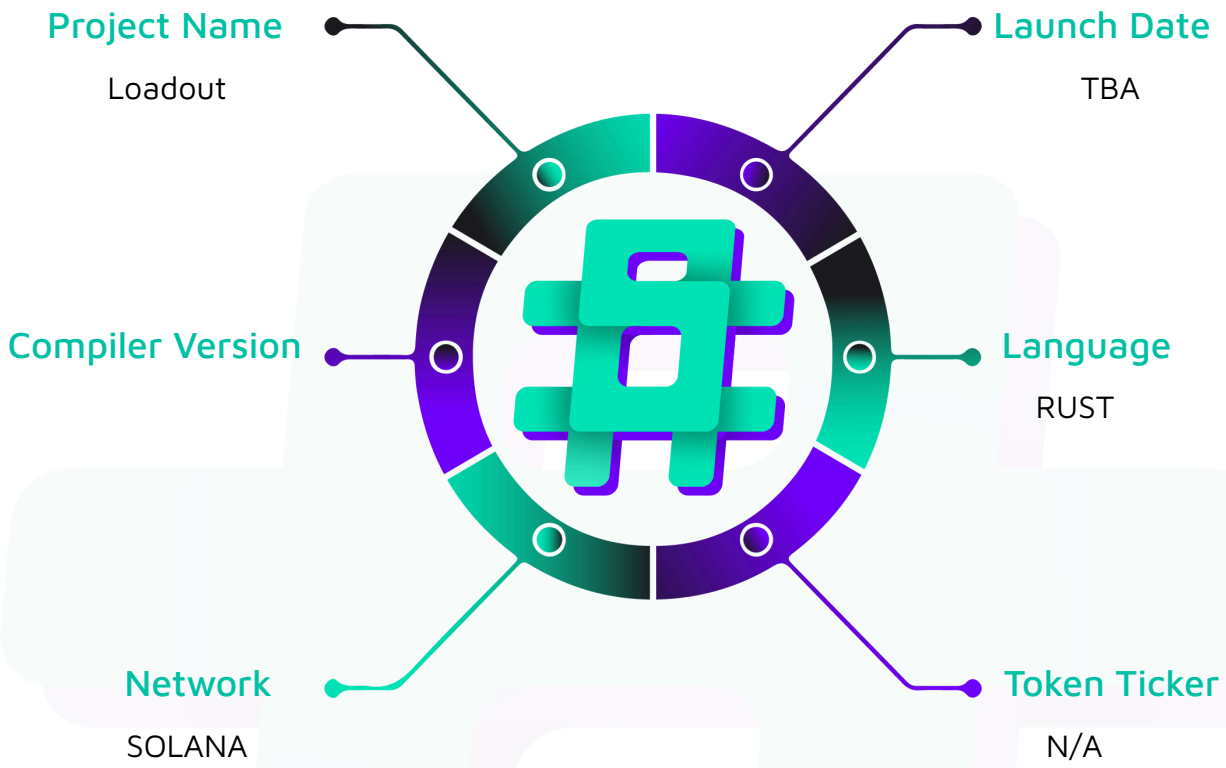
The Loadout team partnered with Hashlock to conduct a security audit of their program. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

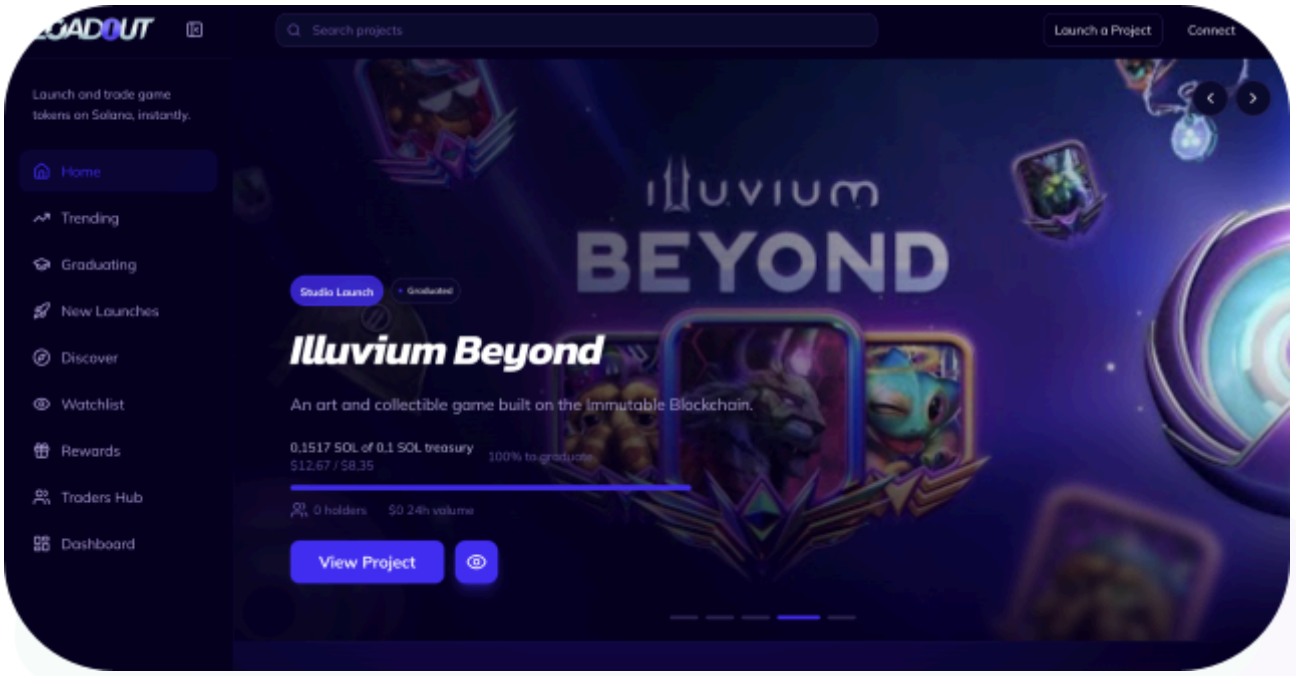
The Loadout Rewards page is part of a broader Web3 gaming platform that enables game studios to fund development through community participation and blockchain-based mechanics. In this context, the rewards system functions as an incentive layer where users (players or early supporters) can earn benefits—such as priority access, in-game assets, or future value—by engaging with upcoming games, backing projects early, or participating in the ecosystem. Built on the Solana blockchain, the platform emphasizes a "community-funded" model in which demand is created before launch, aligning player incentives with game success and turning engagement into tangible, tradeable rewards.

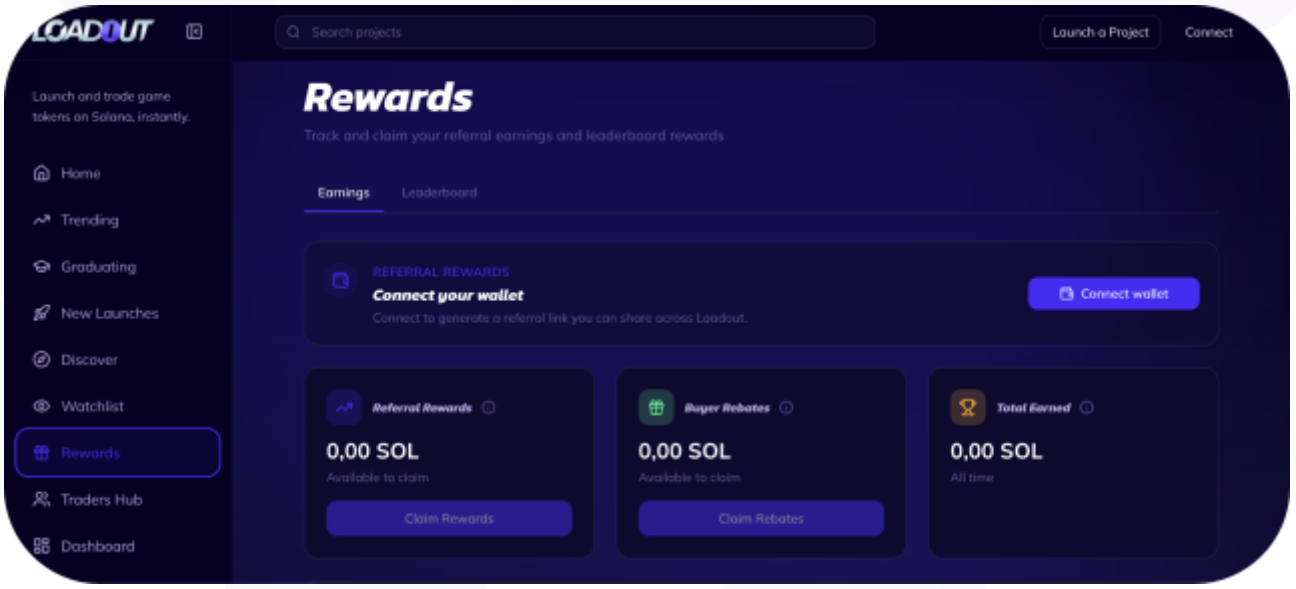
Project Name: Loadout
Project Type: dApp
Website: https://app.loadout.xyz/
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the Rust code within the Loadout project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Loadout Smart Contracts
Platform	Rust
Audit Date	April, 2026
GitHub URL	https://github.com/loadout-core/loadout-programs
In scope files	programs/swap-router/*
Contract 1	constant.rs
Contract 2	error.rs
Contract 3	events.rs
Contract 4	lib.rs
Contract 5	types.rs
Contract 6	mod.rs
Contract 7	proxy_buy.rs
Contract 8	proxy_sell.rs
Audited GitHub Commit Hash	43f5228bbffecac1a754a62fa9a36448ce21c417
Fix Review GitHub Commit Hash	bbe1a8f41f2be4fd8182f3f110ca10d7367b93d2

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

2 Low severity vulnerabilities

6 QAs

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
programs/swap-router/src/instructions/mod.rs - Declares and re-exports `proxy_buy` and `proxy_sell` instruction modules.	**Contract achieves this functionality.**
programs/swap-router/src/instructions/proxy_buy.rs - Defines `ProxyBuy` account context with trader, PDA swap accounts, Meteora pool accounts, fee/project/referrer accounts. - Handler: validates inputs, reads fee config and project data cross-program, computes fees and referral splits, transfers SOL, invokes Meteora DLMM swap CPI, distributes fees to treasury/protocol/referrers/rebate, closes WSOL account, emits `PostGradSwapEvent`.	**Contract achieves this functionality.**
programs/swap-router/src/instructions/proxy_sell.rs - Defines `ProxySell` account context, mirrors ProxyBuy with an additional `swap_token` PDA for the project token. - Handler: transfers project tokens to `swap_token`, invokes Meteora DLMM swap CPI, enforces `min_sol_out` slippage, distributes SOL fees, closes WSOL account, emits `PostGradSwapEvent`.	**Contract achieves this functionality.**
programs/swap-router/src/constants.rs - Defines external program IDs (`LOADOUT_CORE_ID`, `METEORA_DLMM_ID`), PDA seed constants, and discriminator bytes for cross-program account deserialization.	**Contract achieves this functionality.**

programs/swap-router/src/errors.rs - Defines `SwapRouterError` enum with error codes for zero amount, protocol paused, slippage exceeded, invalid accounts, math overflow, etc.	**Contract achieves this functionality.**
programs/swap-router/src/events.rs - Defines `PostGradSwapEvent` emitted on every post-graduation swap with fee breakdown, trader, pool, and referrer details.	**Contract achieves this functionality.**
programs/swap-router/src/lib.rs - Program entrypoint exposing `proxy_buy` and `proxy_sell` instructions, delegates to handlers in the instructions module.	**Contract achieves this functionality.**
programs/swap-router/src/types.rs - Defines cross-program deserialization structs (`FeeConfigData`, `ProjectData`, `ReferrerData`) for reading loadout-core accounts. - Implements compute_referral for referral fee split logic and `invoke_meteora_swap` CPI helper.	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Loadout project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Loadout project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Low

[L-01] programs/swap-router/src/instructions/proxy_buy.rs#proxy_buy - Lamport leak in swap_authority PDA from ephemeral WSOL account rents

Description

In both `proxy_buy` and `proxy_sell`, the WSOL token account (`swap_wsol`) is created via `init_if_needed` with `payer = trader`, then closed after the swap. The `close_account` CPI sends all remaining lamports (including the rent-exempt minimum) to `swap_authority`.

Since the swap-router has no instruction to withdraw lamports from `swap_authority`, these rent lamports are permanently stranded. Every trade incurs a ~0.002 SOL fee per project's `swap_authority` PDA, paid by the trader who funded `init_if_needed`.

Impact

Every successful trade will strand approximately 0.002 SOL on the project's `swap_authority` PDA, funded by the trader. These rent lamports accumulate on the PDA and have no recovery path.

For example, across over 10,000 trades on a single project, roughly 20 SOL will be permanently locked.

Recommendation

In `proxy_buy`, consider changing the `token::CloseAccount` destination for `swap_wsol` from `swap_authority` to `trader`.

In `proxy_sell`, after all distributions are complete, consider transferring excess lamports on `swap_authority` above its rent-exempt minimum back to the `trader`.

Status

Resolved

[L-02] programs/swap-router/src/instructions/proxy_sell.rs#proxy_sell - swap_token account is never closed after the sell

Description

The PDA token account `swap_token` is created via `init_if_needed` in `proxy_sell`, but is never closed at the end of the instruction. In contrast, `swap_wsol` is closed after each trade in both instructions.

Impact

- Rent-exempt lamports (~0.002 SOL) for `swap_token` are paid by the first trader per project and locked permanently.
- If Meteora ever returns fewer tokens than the `amount_in` (e.g., under extreme liquidity conditions), token dust accumulates in `swap_token` and can be extracted by the next trader for the same project.

Recommendation

Consider closing `swap_token` to the trader after the Meteora swap, mirroring the `swap_wsol` pattern.

Status

Resolved

QA

[Q-01] programs/swap-router/src/instructions/proxy_buy.rs#ProxyBuy - Ephemeral swap PDAs use `init_if_needed` instead of `init`

Description

The ephemeral swap accounts (`swap_wsol` in both instructions, `swap_token` in `proxy_sell`) use `init_if_needed`. The accounts are closed to prevent residual WSOL from leaking to the next trader.

Since `swap_wsol` is always closed at the end of each trade and `swap_token` should also be closed, these accounts are single-use, and `init` is the correct constraint.

Recommendation

Consider updating all ephemeral swap accounts to use `init`.

Status

Resolved

[Q-02] programs/swap-router/src/instructions/proxy_sell.rs#ProxySell - Missing token::mint constraint on trader_token_account

Description

In `ProxySell`, `trader_token_account` is declared with only `#[account(mut)]` and no `token::mint` constraint. The equivalent field in `ProxyBuy` binds `token::mint = token_mint`.

Recommendation

Consider adding `token::mint = token_mint` to `trader_token_account` in `ProxySell`.

Status

Resolved

[Q-03] programs/swap-router/src/instructions/proxy_buy.rs#ProxyBuy - No explicit token::authority constraint on trader_token_account

Description

Neither `ProxyBuy` nor `ProxySell` constrains `trader_token_account.authority == trader`.

Recommendation

Consider adding `token::authority = trader` to `trader_token_account` in both instructions for defense in depth.

Status

Resolved

[Q-04] programs/swap-router/src/types.rs#compute_referral - Terse variable names obscure fee split logic

Description

The `split_referral` function returns a tuple destructured as `(rr, br, ap)`, which is hard to read and undermines code readability.

Recommendation

Consider destructing the return values as full names, e.g., `let (referrer_reward, buyer_rebate, adjusted_protocol_fee) = ...`.

Status

Resolved

[Q-05] programs/swap-router/src/instructions/proxy_buy.rs#handler - Optional referrer transfers silently skip when reward is non-zero

Description

The L1/L2 referrer reward transfers are gated on `reward > 0 && account.is_some()`. When `compute_referral` returns a non-zero reward, the referrer account is guaranteed to be `Some`, so the `None` branch is unreachable. The same pattern appears in `proxy_sell`.

This is problematic because silently skipping over `None` values could mask potential invariant-breaking bugs. If `compute_referral` is refactored and the precondition decouples, the reward would be silently discarded rather than failing loudly.

Recommendation

When `l1_reward > 0` or `l2_reward > 0`, consider implementing `require!` so the matching account is `Some`.

Status

Resolved

[Q-06] programs/swap-router/src/instructions/proxy_sell.rs#ProxySell - Non-intuitive swap_token field name

Description

The project-token PDA is named `swap_token` and refers to the swap authority's account for the swap token. The name `swap_token` is ambiguous, as "token" is not a currency name and could be read as a verb.

Recommendation

Consider renaming `swap_token` to `swap_token_account`.

Status

Resolved

Centralisation

The Loadout project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Loadout project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved . Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au